ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ И КОМПЈУТЕРСКО ИНЖЕНЕРСТВО

Algorithms techniques

Algorithms and data structures

Exercise 3

Brute force technique

- The brute force technique solves the problem in the simplest, the most direct and obvious way
- As a result, for this type of algorithms there is bigger complexity in time and space than some smarter or more sophisticated algorithms
- However, brute force algorithms are easy for implementation
- All possibilities are considered

Problem 1

- Given N points (N >= 2) in two-dimensional space, calculate the smallest distance between two points.

Problem 1 - Java

```
class Point {
    double x;
    double y;

    Point(double x, double y) {
        this.x = x;
        this.y = y;
    }
}

class BruteForce {
    int INF = 1000000;

    double min(double a, double b) {
        if (a < b) {
            return a;
        } else {
            return b;
        }
    }
```

Problem 1 - Java

```
    double rastojanie(Point a, Point b) {
        return Math.sqrt((a.x - b.x) * (a.x - b.x) +
                         (a.y - b.y) * (a.y - b.y));
    }

    double najmalo_rastojanie(Point p[], int n) {
        int i, j;
        double best = INF;

        for (i = 0; i < (n - 1); i++) {
            for (j = i + 1; j < n; j++) {
                best = Math.min(best, rastojanie(p[i], p[j]));
            }
        }

        return best;
    }
}
```

Problem 2

- For a given chess board calculate the number of different ways two queens can be set on the board without attacking each other. Two queens attack each other if they are in the same row, column or diagonal.

Problem 2 - Java

```
class BruteForce {


    int dali_se_napagaat(int i1, int j1, int i2, int j2) {
        if (i1 == i2) // ista redica
            return 1;
        if (j1 == j2) // ista kolona
            return 1;
        if (Math.abs(i1 - i2) == Math.abs(j1 - j2)) // ista dijagonala
            return 1;
        return 0;
    }
```

Problem 2 - Java

```
    int broj_na_nacini() {
        int i1, j1, i2, j2;
        int rezultat = 0;

        for (i1 = 0; i1 < 8; i1++) {
            for (j1 = 0; j1 < 8; j1++) {
                for (i2 = 0; i2 < 8; i2++) {
                    for (j2 = 0; j2 < 8; j2++) {
                        if (dali_se_napagaat(i1, j1, i2, j2)==0) {
                            rezultat++;
                        }
                    }
                }
            }
        }

        return rezultat;
    }
}
```

Greedy algorithms

- Archetype "Greedy algorithm" is an algorithm that uses some heuristics to find the optimal solution of a given problem.
- It is based on an algorithm in phases, considering only one variable at the given moment. In each phase a local optimal choice is decided hoping for the global optimal solution.
- Greedy algorithms are mostly faster than brute force techniques.
- Unfortunately, not always greedy strategy gives the correct solution

Greedy algorithms

- Greedy algorithms elements
  - **Greedy choice:** “Local optimum" in the greedy choice gives an optimal solution. At each step the choice is independent from the solutions for the future sub-problem. The idea is that a global optimal solution can be achieved with a local optimal choice. Usually the greedy strategy is top-down, with greedy choices one after the other, iteratively decreasing the problem.

Greedy algorithms

- Greedy algorithms elements:
  - **Optimal substructure**: the optimal solution can be achieved gathering the optimal sub-problems solutions. A given problem has an optimal sub-structure if the optimal solution can be achieved gathering the optimal solutions of the sub-problems. A given problem has an optimal problem sub-structure if the optimal solution contains the sub-problems optimal solutions.

Problem 3

- There are coins with some specific value (there are coins of 50, 10, 5, 2 and 1 denar), and the number of coins of each value is unlimited. For a given sum calculate the smallest number of coins for forming that sum.

Problem 3 - solution

- For solving this problem we can use the archetype „Greedy algorithm“, so that we always take a bigger number from the biggest coins. At first, the greedy choice is consisted of taking the biggest number of coins of the biggest coin value possible (local optimum) to form a sum closer to the given sum. This coins choice at the moment looks like the best choice and the number of those biggest coins is added to the solution. Afterwards, for the rest of the sum, or the sub-problem we continue with the next biggest coin value and again we take as much coins as possible. In the same manner, we continue with the coins sorted by value until the given sum is formed.

Problem 3 - Java

```
class Greedy {

    void sortiraj_paricki(int coins[], int n) {
        int i, j, tmp;

        for (i = 0; i < n; i++) {
            for (j = i + 1; j < n; j++) {
                if (coins[i] < coins[j]) {
                    tmp = coins[i];
                    coins[i] = coins[j];
                    coins[j] = tmp;
                }
            }
        }
    }
```

Problem 3 - Java

```
// coins e niza so vrednostite na parickite koi se dadeni
// n e brojot na paricki
// sum e dadenata suma
// coinsNum e niza za resenieto so brojot na paricki od sekoja
// golemina na paricka
int greedyCoins(int coins[], int n, int sum, int coinsNum[]) {
    sortiraj_paricki(coins, n);

    int i = 0;
    int br = 0; // vkupniot broj na paricki za da se formira
                // dadenata suma

    while (sum > 0) {
        coinsNum[i] = sum / coins[i];
        sum -= coinsNum[i] * coins[i];
        br += coinsNum[i];
        i++;
    }
```

Problem 3 - Java

```
        while (i < n) {
            coinsNum[i] = 0;
            i++;
        }

        return br;
    }
}
```

Problem 3 - discussion

- Does this algorithm gives an optimal solution always? What is the minimal number of coins for forming a sum of 13 denars, if we have coins of 1, 2, 5, 8, 10 denars?
- Is the brute force solution optimal (problem 3)?

Knapsack problem

- Mathematical formulation
  - Let w_i be the weight of the i-th object
  - Let p_i be the profit if the i-th object is taken
  - Let C be the knapsack capacity
  - Let x_i be a variable with value
    - 0 or 1 for "0/1 knapsack problem". $x_i = 1$ means that the i–th object is taken
    - $0 \leq x_i \leq 1$ for "fractional knapsack problem". A fractional part of the object can be taken

Knapsack problem

- Aim and constraints
  - For given $\{w_1, w_2, \ldots, w_n\}$ and $\{p_1, p_2, \ldots, p_n\}$
  - Aim: maximize $\sum_{i=1}^{n} p_i x_i$
  - Constraint: $\sum_{i=1}^{n} w_i x_i \leq C$

Problem 4

- A student goes on a trip. He is allowed to take a suitcase with maximum weight 20kg. The student has to decide what to take: clothes, books or some games. After measuring, the following table is given:

#	Object	Value/ profit	Weigth
1	Clothes	200	10
2	Books	150	20
3	Monopol, cards…	5	0.5
4	CD player, MP3 player…	80	5

Problem 4

- What should the student take, so that the profit of those things is maximal?
- Two versions of this problem:
  - **Fractional knapsack problem**: The student can take object parts i.e. the student can decide to take only one part x_i for the object o_i, where $0 \leq x_i \leq 1$.
  - **0-1 knapsack problem**: the problem is set the same way, but the objects can not be divided in smaller parts, i.e. the students has to decide to take the object or not. That means a part of the object can not be taken (x_i=0 or x_i=1).

Problem 4 (fractional knapsack) – analysis

- To solve the Fractional knapsack problem we can use the archetype “Greedy algorithm".
- For example, let the following problem be given: the number of objects is 3 (n=3), the capacity of the knapsack is C=20, the profit of each of the objects is given with the following vector (p1, p2, p3) = (25, 24, 15) and the weights are (t1, t2, t3) = (18, 15, 10).Fill the knapsack with objects to gain the maximum profit.

Problem 4 (fractional knapsack) - Java

```
class Greedy {

    void sortiraj(int p[], int t[], int n) {
        int i, j, tmpP, tmpT;

        for (i = 0; i < n; i++) {
            for (j = i + 1; j < n; j++) {
                if ((p[i]/(float)t[i])<(p[j]/(float) t[j])) {
                    tmpP = p[i];
                    tmpT = t[i];
                    p[i] = p[j];
                    t[i] = t[j];
                    p[j] = tmpP;
                    t[j] = tmpT;
                }
            }
        }

    }
```

Problem 4 (fractional knapsack) - Java

```
// p i t gi sodrzat profitot i tezinata na objektite
// C e kapacitet na paketot, x e vektor na resenieto
float grFractKnp(int p[], int t[], float C, int n, float x[]) {

    sortiraj(p, t, n);
    // objektite se podredeni taka da bide zadovolen
       // p[i]/t[i] >= p[i+1]/t[i+1]

    int i;
    float profit = 0;

    for (i = 0; i < n; i++) {
        x[i] = 0;
    }
```

Problem 4 (fractional knapsack) - Java

```
    for (i = 0; i < n; i++) {
        if (C > t[i]) {          // objektot go stavame celosno
           x[i] = 1;
           C -= t[i];
            profit += p[i];
        } else {                 // objektot go stavame delumno
            x[i] = (C / (float) t[i]);
               profit += x[i] * (float) p[i];
            C = 0;
            break;
        }
    }

    return profit;
}
```

Problem 4 (0-1 knapsack) - discussion

- Before using the greedy algorithm we should be sure that the function that chooses the next object will lead to the optimal solution. Fortunately, in many cases it is possible to find the optimal solution with a greedy algorithm. But, for the 0-1 knapsack problem a greedy algorithm can not be used. The optimal solution is with another algorithm ☺.

Divide and conquer

- The archetype algorithm "Divide and conquer“ is one of the most powerful techniques for solving problems that can be divided into smaller simpler solvable parts. Recursion usage is obvious and provides problem details.
- “Divide and conquer" algorithm divides the problem into smaller sub-problems, solves each of the sub-problems and in the end merges the solutions into one solution of the problem. Example problems that use “Divide and conquer“ are sorting algorithms: quicksort and mergesort.

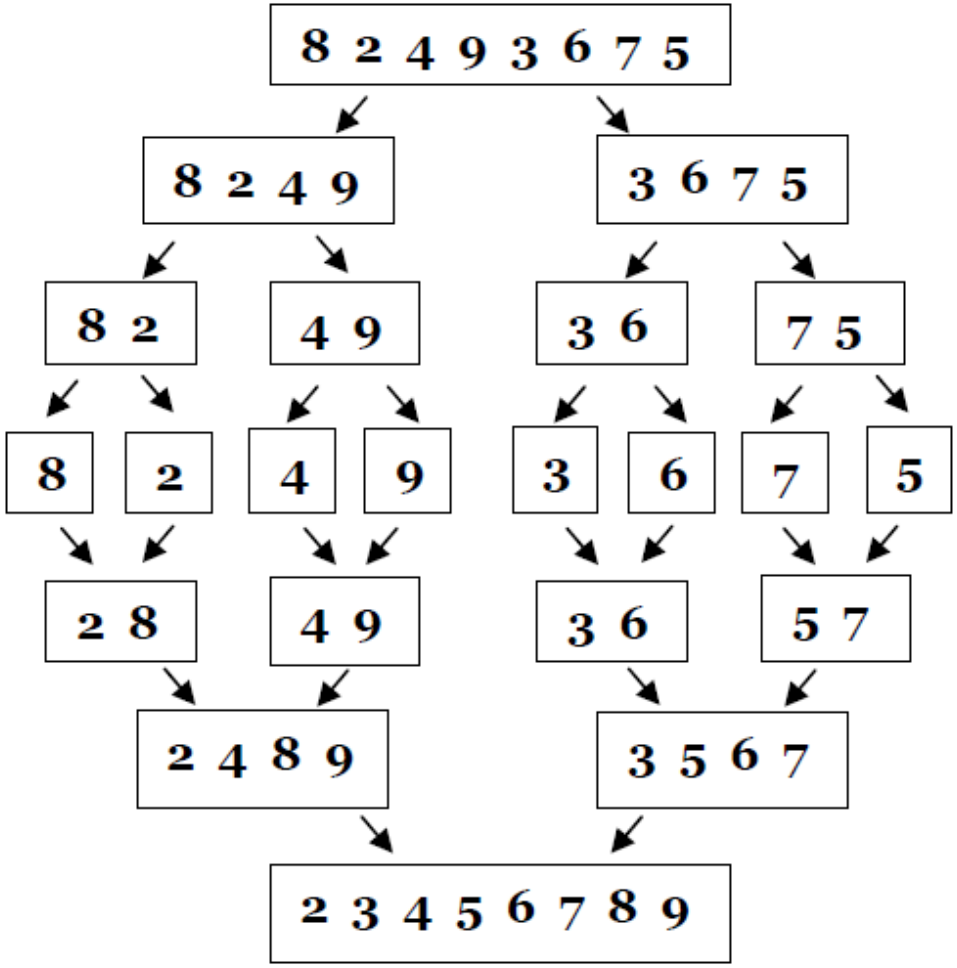
Divide and conquer

- Algorithm:
  - **Divide**: divide the problem into sub-problems P_1, P_2, … , P_n till you get to a basic case
  - **Conquer**: solve problems P_i (use a recursion), getting solutions R_i
  - **Merge**: combine problems solutions R_i for the problem solution

Merge sort

- “Divide and conquer" algorithm in 3 steps for mergesort of an array [0 … n]:
  - **Divide**: the array a[0..n] is halved into two subarrays
  - **Conquer**: each half a[0 .. (n/2)] and a[(n/2) +1 .. n] is sorted recursively
  - **Merge**: both sorted halves are merged into a sorted array

Merge sort


8 2 4 9 3 6 7 5
8 2 4 9
3 6 7 5
8 2
4 9
3 6
7 5
8
2
4
9
3
6
7
5
2 8
4 9
3 6
5 7
2 4 8 9
3 5 6 7
2 3 4 5 6 7 8 9

Merge sort - Java

```
class DivideAndConquer {
    //spojuvanje na dve sortirani nizi [l, mid], [mid+1, r]
    //rezultatot e nova sortirana niza
    void merge(int a[], int l, int mid, int r) {
        int numel = r - l + 1;
        int temp[] = new int[100]; // nova niza za privremeno cuvanje
                                   // na sortiranite elementi
        int I = l, j = mid+1, k = 0;

        while ((i <= mid) && (j <= r)) {
            if (a[i] < a[j]) {
                temp[k] = a[i];
                i++;
            } else {
                temp[k] = a[j];
                j++;
            }
            k++;
        }
```

Merge sort - Java

```
        while (i <= mid) {
            temp[k] = a[i];
            i++;
            k++;
        }

        while (j <= r) {
            temp[k] = a[j];
            j++;
            k++;
        }

        for (k = 0; k < numel; k++) {
            a[l + k] = temp[k];
        }
    }
```

Merge sort - Java

```
    void mergesort(int a[], int l, int r) {
        if (l == r) {
            return;
        }

        int mid = (l + r) / 2;
        mergesort(a, l, mid);
        mergesort(a, mid + 1, r);
        merge(a, l, mid, r);
    }
}
```

Merge sort - analysis

- Algorithm complexity:

  $T(n) = 2T(n/2)+n$

  $2T(n/2) = 2(2(T(n/4))+n/2) = 4T(n/4)+n \rightarrow$
  $T(n) = 4T(n/4)+2n$

  $4T(n/4) = 4(2T(n/8))+(n/4) = 8T(n/8)+n \rightarrow$
  $T(n) = 8T(n/8)+3n$

  …

  $T(n) = 2^k T(n/ 2^k)+kn$ and with k=log(n):

  $T(n) = nT(1)+n \log(n) = n \log(n)+n$

  Hence, the complexity is $O(n \log_2 n)$

Problem 5

- Write a function that solves the n-th power of a number using “Divide and conquer".

Problem 5 - Java

```
class DivideAndConquer {

   int pow(int x, int n) {
        int r;

        if(n == 0)
            return(1);
        else if(n % 2 == 0) {
            r = pow(x, (n/2));
            return r*r;
        } else {
            r = pow(x, (n/2));
            return x*r*r;
        }
    }
}
```

Problem 5 - analysis

- Complexity analysis of the algorithm:

  $T(n) = T(n/2) + O(1)$

  $T(n/2) = (T(n/4) + 1) \rightarrow$

  $T(n) = T(n/4) + 2$

  $T(n/4) = T(n/8) + 1 \rightarrow$

  $T(n) = T(n/8) + 3$

  ...

  $T(n) = T(n/2^k)+k$ and if $k=\log(n)$ we have: $T(n) = T(1)+\log(n) = \log(n)$

  It means that complexity is *O(log n)*

Problem 5 - analysis

- Solution 2:

```
int pow(int x, int n) {
    int r;

    if(n == 0)
        return(1);
    else if(n % 2 == 0) {
        return pow(x, (n/2))*pow(x,
(n/2));
    } else {
        return x*pow(x, (n/2))*pow(x,
(n/2));
    }
}
```

- What can be concluded?

Problem 5 (solution 2)- analysis

- Complexity analysis of the algorithm :

$T(n) = 2T(n/2)+O(1)$

$T(n/2) = 2(T(n/4) + 1) \rightarrow$

$T(n) = 4T(n/4)+2$

$T(n/4) = 8T(n/8) + 1 \rightarrow$

$T(n) = 8T(n/8)+3$

...

$T(n) = 2^kT(n/2^k)+k$ and if $k=\log(n)$ we have: $T(n) = nT(1)+\log(n) = n + \log(n)$

It means that complexity is *O(n)*